

4.5 ERROR RECOVERY IN TOP-DOWN PARSERS

The response of a parser to syntax errors is often a critical factor in the usefulness of a compiler. Minimally, a parser must determine whether a program is syntactically correct or not. A parser that performs this task alone is called a **recognizer**, since all it does is recognize strings in the context-free language generated by the grammar of the programming language in question. It is worth stating that at a minimum, any parser must behave like a recognizer—that is, if a program contains a syntax error, the parser must indicate that *some* error exists, and conversely, if a program contains no syntax errors, then the parser should not claim that an error exists.

Beyond this minimal behavior, a parser can exhibit many different levels of responses to errors. Usually, a parser will attempt to give a meaningful error message, at least for the first error encountered, and also attempt to determine as closely as possible the location where that error has occurred. Some parsers may go so far as to attempt some form of **error correction** (or, perhaps more appropriately, **error repair**), where the parser attempts to infer a correct program from the incorrect one given. If this is attempted, most of the time it is limited to easy cases, such as missing punctuation. There exists a body of algorithms that can be applied to find a correct program that is closest in some sense to the one given (usually in terms of number of tokens that must be inserted, deleted, or changed). Such **minimal distance error correction** is usually too inefficient to be applied to every error and, in any case, results in error repair that is often very far from what the programmer actually intended. Thus, it is seldom seen in actual parsers. Compiler writers find it difficult enough to generate meaningful error messages without trying to do error correction.

Most of the techniques for error recovery are ad hoc, in the sense that they apply to specific languages and specific parsing algorithms, with many special cases for individual situations. General principles are hard to come by. Some important considerations that apply are the following.

1. A parser should try to determine that an error has occurred *as soon as possible*. Waiting too long before declaring error means the location of the actual error may have been lost.
2. After an error has occurred, the parser must pick a likely place to resume the parse. A parser should always try to parse as much of the code as possible, in order to find as many real errors as possible during a single translation.
3. A parser should try to avoid the **error cascade problem**, in which one error generates a lengthy sequence of spurious error messages.
4. A parser must avoid infinite loops on errors, in which an unending cascade of error messages is generated without consuming any input.

Some of these goals conflict with each other, so that a compiler writer is forced to make trade-offs during the construction of an error handler. For example, avoiding the error cascade and infinite loop problems can cause the parser to skip some of the input, compromising the goal of processing as much of the input as possible.

4.5.1 Error Recovery in Recursive-Descent Parsers

A standard form of error recovery in recursive-descent parsers is called **panic mode**. The name derives from the fact that, in complex situations, the error handler will con-

sume a possibly large number of tokens in an attempt to find a place to resume parsing (in the worst case, it might even consume the entire rest of the program, in which case it is no better than simply exiting after the error). However, when implemented with some care, this can be a much better method for error recovery than its name implies.⁴ Panic mode has the added attraction that it virtually ensures that the parser cannot get into an infinite loop during error recovery.

The basic mechanism of panic mode is to provide each recursive procedure with an extra parameter consisting of a set of **synchronizing tokens**. As parsing proceeds, tokens that may function as synchronizing tokens are added to this set as each call occurs. If an error is encountered, the parser **scans ahead**, throwing away tokens until one of the synchronizing set of tokens is seen in the input, whence parsing is resumed. Error cascades are avoided (to a certain extent) by not generating new error messages while this forward scan takes place.

The important decisions to be made in this error recovery method are what tokens to add to the synchronizing set at each point in the parse. Generally, Follow sets are important candidates for such synchronizing tokens. First sets may also be used to prevent the error handler from skipping important tokens that begin major new constructs (like statements or expressions). First sets are also important, in that they allow a recursive descent parser to detect errors early in the parse, which is always helpful in any error recovery. It is important to realize that panic mode works best when the compiler “knows” when *not* to panic. For example, missing punctuation symbols such as semicolons or commas, and even missing right parentheses, should not always cause an error handler to consume tokens. Of course, care must be taken to ensure that an infinite loop cannot occur.

We illustrate panic mode error recovery by sketching in pseudocode its implementation in the recursive descent calculator of Section 4.1.2 (see also Figure 4.1). In addition to the *match* and *error* procedures, which remain essentially the same (except that *error* no longer exits immediately), we have two more procedures, *checkinput*, which performs the early lookahead checking, and *scanto*, which is the panic mode token consumer proper:

```

procedure scanto ( synchset ) ;
begin
    while not ( token in synchset  $\cup$  { $ } ) do
        getToken ;
    end scanto ;

procedure checkinput ( firstset, followset ) ;
begin
    if not ( token in firstset ) then
        error ;
        scanto ( firstset  $\cup$  followset ) ;
    end if ;
end ;

```

Here the \$ refers to the end of input (EOF).

4. Wirth [1976], in fact, calls panic mode the “don’t panic” rule, presumably in an attempt to improve its image.

These procedures are used as follows in the *exp* and *factor* procedures (which now take a *synchset* parameter):

```

procedure exp ( synchset ) ;
begin
  checkinput ( { (, number }, synchset ) ;
  if not ( token in synchset ) then
    term ( synchset ) ;
    while token = + or token = - do
      match ( token ) ;
      term ( synchset ) ;
    end while ;
    checkinput ( synchset, { (, number } ) ;
  end if;
end exp ;

procedure factor ( synchset ) ;
begin
  checkinput ( { (, number }, synchset ) ;
  if not ( token in synchset ) then
    case token of
      ( : match( ( ) ;
        exp ( { } ) ;
        match( ) ) ;
      number :
        match(number) ;
    else error ;
    end case ;
    checkinput ( synchset, { (, number } ) ;
  end if ;
end factor ;

```

Note how *checkinput* is called twice in each procedure: once to verify that a token in the First set is the next token in the input and a second time to verify that a token in the Follow set (or *synchset*) is the next token on exit.

This form of panic mode will generate reasonable errors (useful error messages can also be added as parameters to *checkinput* and *error*). For example, the input string $(2+-3)*4-+5$ will generate exactly two error messages (one at the first minus sign and one at the second plus sign).

We note that, in general, *synchset* is to be passed down in the recursive calls, with new synchronizing tokens added as appropriate. In the case of *factor*, an exception is made after a left parenthesis is seen: *exp* is called with right parenthesis only as its follow set (*synchset* is discarded). This is typical of the kind of ad hoc analysis that accompanies panic mode error recovery. (We did this so that, for example, the expression $(2+*)$ would not generate a spurious error message at the right parenthesis.) We leave an analysis of the behavior of this code, and its implementation in C, to the exercises. Unfortunately, to obtain the best error messages and error recovery, virtually every token test must be examined for the possibility that a more general test, or an earlier test, may improve error behavior.

4.5.2 Error Recovery in LL(1) Parsers

Panic mode error recovery can be implemented in LL(1) parsers in a similar manner to the way it is implemented in recursive-descent parsing. Since the algorithm is nonrecursive, a new stack is required to keep the *synchset* parameters, and a call to *checkinput* must be scheduled by the algorithm before each generate action of the algorithm (when a nonterminal is at the top of the stack).⁵ Note that the primary error situation occurs with a nonterminal A at the top of the stack and the current input token is not in $\text{First}(A)$ (or $\text{Follow}(A)$, if ϵ is in $\text{First}(A)$). The case where a token is at the top of the stack, and it is not the same as the current input token, does not normally occur, since tokens are, in general, only pushed onto the stack when they are actually seen in the input (table compression methods may compromise this slightly). We leave the modifications to the parsing algorithm of Figure 4.2, page 155, to the exercises.

An alternative to the use of an extra stack is to statically build the sets of synchronizing tokens directly into the LL(1) parsing table, together with the corresponding actions that *checkinput* would take. Given a nonterminal A at the top of the stack and an input token that is not in $\text{First}(A)$ (or $\text{Follow}(A)$, if ϵ is in $\text{First}(A)$), there are three possible alternatives:

1. Pop A from the stack.
2. Successively pop tokens from the input until a token is seen for which we can restart the parse.
3. Push a new nonterminal onto the stack.

We choose alternative 1 if the current input token is $\$$ or is in $\text{Follow}(A)$ and alternative 2 if the current input token is not $\$$ and is not in $\text{First}(A) \cup \text{Follow}(A)$. Option 3 is occasionally useful in special situations, but is rarely appropriate (we shall discuss one case shortly). We indicate the first action in the parsing table by the notation *pop* and the second by the notation *scan*. (Note that a *pop* action is equivalent to a reduction by an ϵ -production.)

With these conventions, the LL(1) parsing table (Table 4.4) looks as in Table 4.9. The behavior of an LL(1) parser using this table, given the string $(2+*)$, is shown in Table 4.10. In that table, the parse is shown only from the first error on (so the prefix $(2+$ has already been successfully matched). We also use the abbreviations E for *exp*, E' for *exp'*, and so on. Note that there are two adjacent error moves before the parse resumes successfully. We can arrange to suppress an error message on the second error move by requiring, after the first error, that the parser make one or more successful moves before generating any new error messages. Thus, error message cascades would be avoided.

There is (at least) one problem in this error recovery method that calls for special action. Since many error actions pop a nonterminal from the stack, it is possible for the stack to become empty, with some input still to parse. A simple case of this in the example just given is any string beginning with a right parenthesis: this will cause E to be immediately popped, leaving the stack empty with all the input yet to be consumed.

5. Calls to *checkinput* at the end of a match, as in the recursive-descent code, can also be scheduled by a special stack symbol in a similar fashion to the value computation scheme of Section 4.2.4, page 167.

One possible action that the parser can take in this situation is to push the start symbol onto the stack and scan forward in the input until a symbol in the First set of the start symbol is seen.

Table 4.9

LL(1) parsing table (Table 4.4) with error recovery entries

$M[N, T]$	(	number	)	+	-	*	\$
<i>exp</i>	$exp \rightarrow$ <i>term exp'</i>	$exp \rightarrow$ <i>term exp'</i>	pop	scan	scan	scan	pop
<i>exp'</i>	scan	scan	$exp' \rightarrow \epsilon$	$exp' \rightarrow$ <i>addop</i> <i>term exp'</i>	$exp' \rightarrow$ <i>addop</i> <i>term exp'</i>	scan	$exp' \rightarrow \epsilon$
<i>addop</i>	pop	pop	scan	$addop \rightarrow$ +	$addop \rightarrow$ -	scan	pop
<i>term</i>	$term \rightarrow$ <i>factor</i> <i>term'</i>	$term \rightarrow$ <i>factor</i> <i>term'</i>	pop	pop	pop	scan	pop
<i>term'</i>	scan	scan	$term' \rightarrow \epsilon$	$term' \rightarrow \epsilon$	$term' \rightarrow \epsilon$	$term' \rightarrow$ <i>mulop</i> <i>factor</i> <i>term'</i>	$term' \rightarrow \epsilon$
<i>mulop</i>	pop	pop	scan	scan	scan	$mulop \rightarrow *$	pop
<i>factor</i>	$factor \rightarrow$ (<i>exp</i>)	$factor \rightarrow$ number	pop	pop	pop	pop	pop

Table 4.10

Moves of an LL(1) parser
using Table 4.9

Parsing stack	Input	Action
\$ <i>E'</i> <i>T'</i>) <i>E'</i> <i>T</i>	*) \$	scan (error)
\$ <i>E'</i> <i>T'</i>) <i>E'</i> <i>T</i>	) \$	pop (error)
\$ <i>E'</i> <i>T'</i>) <i>E'</i>	) \$	$E' \rightarrow \epsilon$
\$ <i>E'</i> <i>T'</i>)	) \$	match
\$ <i>E'</i> <i>T'</i>	\$	$T' \rightarrow \epsilon$
\$ <i>E'</i>	\$	$E' \rightarrow \epsilon$
\$	\$	accept

4.5.3 Error Recovery in the TINY Parser

The error handling of the TINY parser, as given in Appendix B, is extremely rudimentary: only a very primitive form of panic mode recovery is implemented, without the synchronizing sets. The **match** procedure simply declares error, stating which token it found that it did not expect. In addition, the procedures **statement** and **factor** declare error when no correct choice is found. Also the **parse** procedure declares error

if a token other than end of file is found after the parse finishes. The principal error message generated is “unexpected token,” which can be very unhelpful to the user. In addition, the parser makes no attempt to avoid error cascades. For example, the sample program with a semicolon added after the write-statement

```
...
5: read x ;
6: if 0 < x then
7:   fact := 1;
8:   repeat
9:     fact := fact * x;
10:    x := x - 1
11:  until x = 0;
12:  write fact; {<- -- BAD SEMICOLON! }
13: end
14:
```

causes the following *two* error messages to be generated (when only one error has occurred):

```
Syntax error at line 13: unexpected token -> reserved word: end
Syntax error at line 14: unexpected token -> EOF
```

And the same program with the comparison < deleted in the second line of code

```
...
5: read x ;
6: if 0 x then { <- -- COMPARISON MISSING HERE! }
7:   fact := 1;
8:   repeat
9:     fact := fact * x;
10:    x := x - 1
11:  until x = 0;
12:  write fact
13: end
14:
```

causes *four* error messages to be printed in the listing:

```
Syntax error at line 6: unexpected token -> ID, name = x
Syntax error at line 6: unexpected token -> reserved word: then
Syntax error at line 6: unexpected token -> reserved word: then
Syntax error at line 7: unexpected token -> ID, name = fact
```

On the other hand, some of the TINY parser’s behavior is reasonable. For example, a missing (rather than an extra) semicolon will generate only one error message, and the parser will go on to build the correct syntax tree as if the semicolon had been there all along, thus performing a rudimentary form of error correction in this one case. This behavior results from two coding facts. The first is that the **match** procedure does not

consume a token, which results in behavior that is identical to inserting a missing token. The second is that the **stmt_sequence** procedure has been written so as to connect up as much of the syntax tree as possible in the case of an error. In particular, care was taken to make sure that the sibling pointers are connected up whenever a non-nil pointer is found (the parser procedures are designed to return a nil syntax tree pointer if an error is found). Also, the obvious way of writing the body of **stmt_sequence** based on the EBNF

```
statement();
while (token==SEMI)
{ match(SEMI);
  statement();
}
```

is instead written with a more complicated loop test:

```
statement();
while ((token!=ENDFILE) && (token!=END) &&
      (token!=ELSE) && (token!=UNTIL))
{ match(SEMI);
  statement();
}
```

The reader may note that the four tokens in this negative test comprise the Follow set for *stmt_sequence*. This is not an accident, as a test may either look for a token in the First set (as do the procedures for *statement* and *factor*) or look for a token *not* in the Follow set. This latter is particularly effective in error recovery, since if a First symbol is missing, the parse would stop. We leave to the exercises a trace of the program behavior to show that a missing semicolon would indeed cause the rest of the program to be skipped if **stmt_sequence** were written in the first form given.

Finally, we note that the parser has also been written in such a way that it cannot get into an infinite loop when it encounters errors (the reader should have worried about this when we noted that **match** does not consume an unexpected token). This is because, in an arbitrary path through the parsing procedures, eventually either the default case of **statement** or **factor** must be encountered, and both of these *do* consume a token while generating an error message.

EXERCISES

- 4.1 Write pseudocode for *term* and *factor* corresponding to the pseudocode for *exp* in Section 4.1.2 (page 150) that constructs a syntax tree for simple arithmetic expressions.
- 4.2 Given the grammar $A \rightarrow (A)A \mid \varepsilon$, write pseudocode to parse this grammar by recursive-descent.
- 4.3 Given the grammar

```
statement  $\rightarrow$  assign-stmt | call-stmt | other
assign-stmt  $\rightarrow$  identifier := exp
call-stmt  $\rightarrow$  identifier ( exp-list )
```

write pseudocode to parse this grammar by recursive-descent.